

DATA MINING & MACHINE LEARNING

Өгөгдөл олборлолт ба
машин сургалт
S.DMM213

Б.Золзаяа, Ph.D
Мэдээллийн технологийн тэнхим
МХТС, ШУТИС



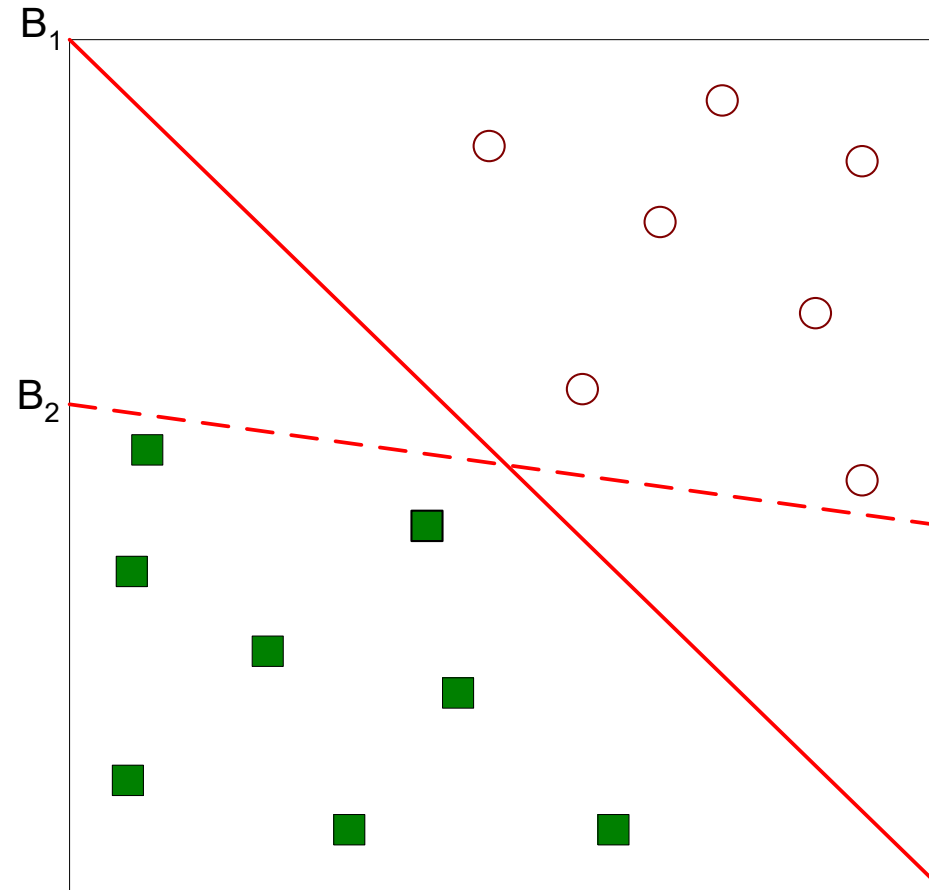
Агуулга

- **Ангилагчууд**
 - Support vector machines (*SVM*)
 - Хиймэл мэдрэлийн сүлжээ / *Artificial neural network (ANN)*

Support Vector Machines (SVM)

- *SVM* нь *шугаман* болон *шугаман бус* өгөгдлийг хоёуланг нь зохицуулж чаддаг буюу ангилж чаддаг арга.
- *SVM* нь *тоон таамаглал* болон *ангилал* хоёуланд нь хэрэглэгддэг.
- Бусад аргуудтай харьцуулахад *хэт тохируулагдах* нь бага байдаг.
- Дараах хэд хэдэн салбарт ашиглагддаг.
 - *handwritten digit recognition*
 - *object recognition*
 - *speaker identification*

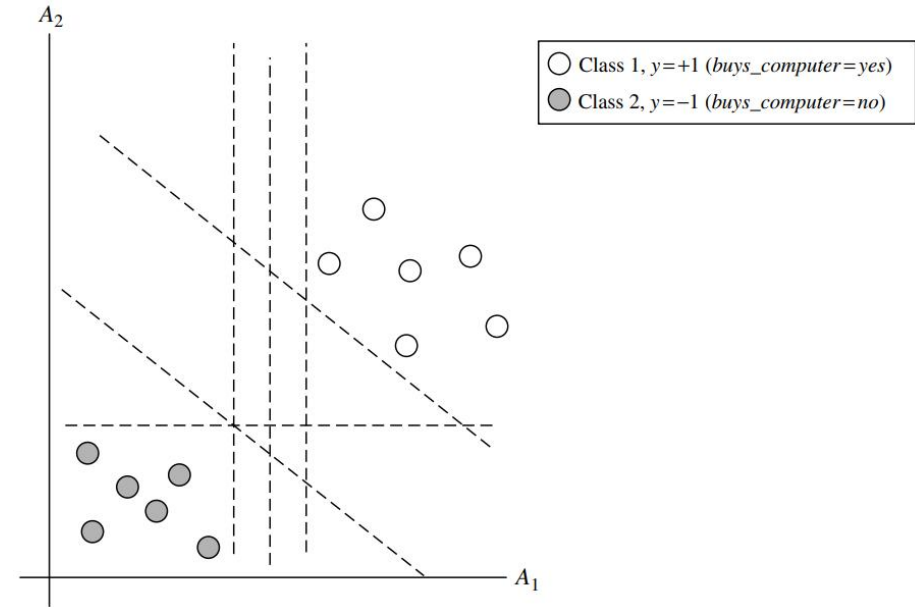
Support Vector Machines (SVM)



- Аль нь дээр вэ? B_1 эсвэл B_2 ?
- Илүү сайныг яаж тодорхойлох вэ?

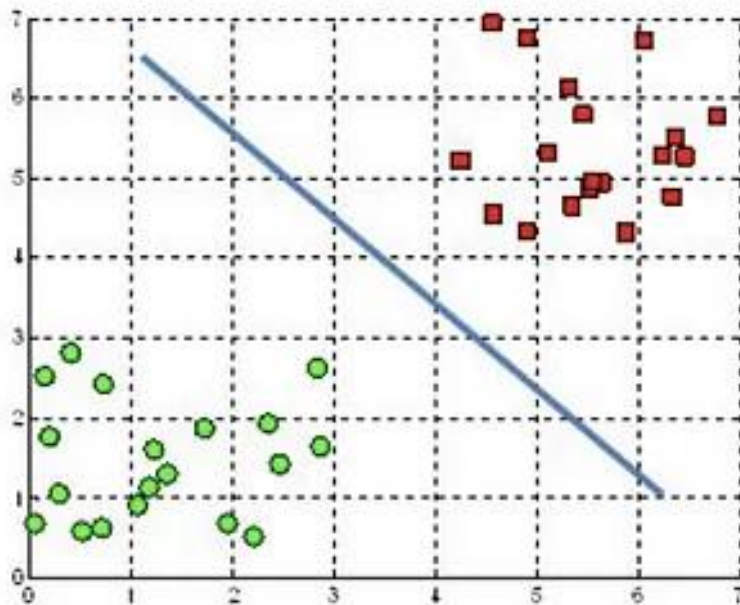
SVM: Шугаман салах боломжтой өгөгдөл

- 2 ангилалтай, $y_i \rightarrow +1$; $-1 \rightarrow buys\ computer = yes$; $buys\ computer = no$
- $D, (X_1, y_1), (X_2, y_2), \dots, (X_{|D|}, y_{|D|})$
- 2 атрибуттай: A_1 болон A_2
- 2 хэмжээст өгөгдөл нь шугаман салах боломжтой.
 - Нэг ангиллын бүх бичлэгүүд нөгөө ангиллын бүх бичлэгүүдээс шулуунаар тусгаарлагдаж байна.
 - Хязгааргүй олон тооны тусгаарлах шулуун
 - Хамгийн сайн шулууныг (*line*) хэрхэн олох вэ?
 - Өмнө харж байгаагүй бичлэгүүд дээр хамгийн бага ангиллын алдаатай байх
 - Хэрэв өгөгдөл 3 хэмжээст ($3D$) бол хамгийн сайн тусгаарлах хавтгайг (*plane*) олох болно.
 - n хэмжээсийн хувьд хамгийн сайн гипер хавтгайг (*hyperplane*) олно.

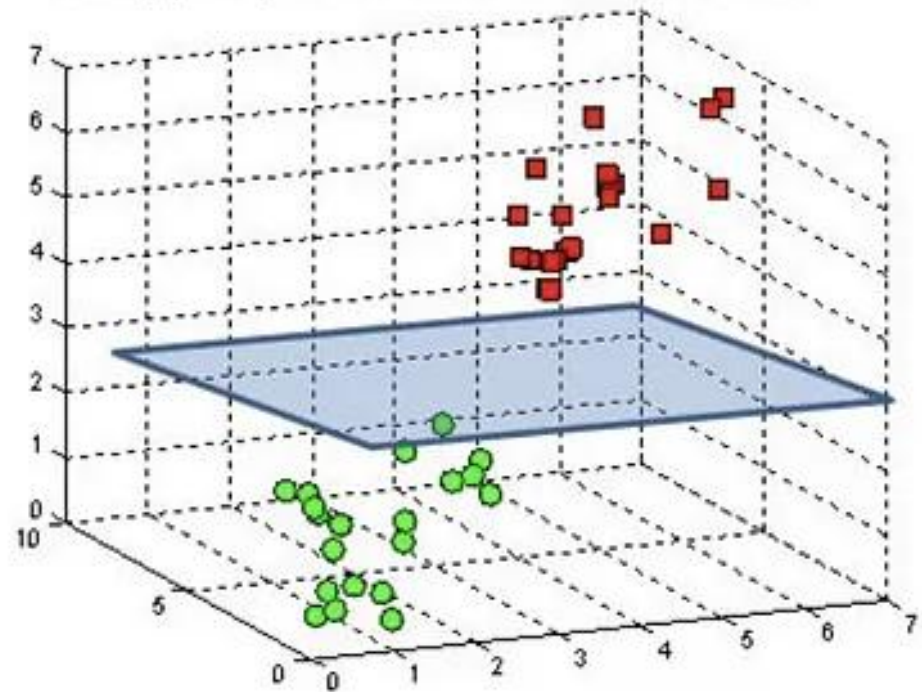


SVM

A hyperplane in $\mathbb{R}^2$ is a line

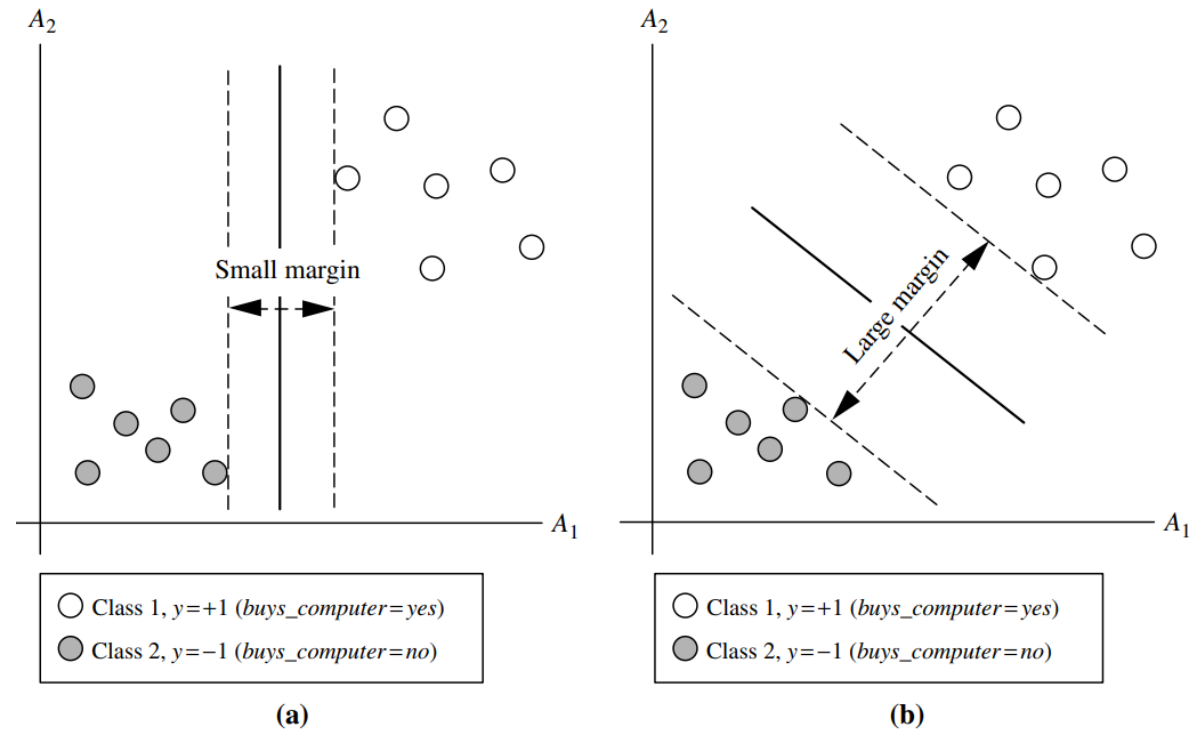


A hyperplane in $\mathbb{R}^3$ is a plane



SVM: Шугаман салах боломжтой өгөгдөл

- Оролтын атрибутын тооноос үл хамааран хайж буй шийдвэрийн хилийг илэрхийлэх гипер хавтгайг (*hyperplane*) цаашид ашиглая.
- Хамгийн сайн гипер хавтгайг хэрхэн олох вэ?
- Хайх: *maximum marginal hyperplane (MMH)*
- Их захын зайтай > Бага захын зайтай
- Ирээдүйн өгөгдлийг ангилахад илүү нарийвчлалтай байна.
- Энэ нь ангиллуудын хоорондох хамгийн том ялгааг өгдөг.



SVM: Шугаман салах боломжтой өгөгдөл

- *ММН*: энэ зай нь үнэндээ *ММН*-ээс аль нэг ангиллын хамгийн ойрын сургалтын бичлэг хүртэлх зай.

$$W \cdot X + b = 0$$

- Тусгаарлах гипер хавтгай:

- W – weight vector $W = \{w_1, w_2, \dots, w_n\}$
- n – атрибутын тоо $(x_1, x_2, \dots, x_n)$
- X - оролтын өгөгдөл (feature vector)
- b - хазайлт

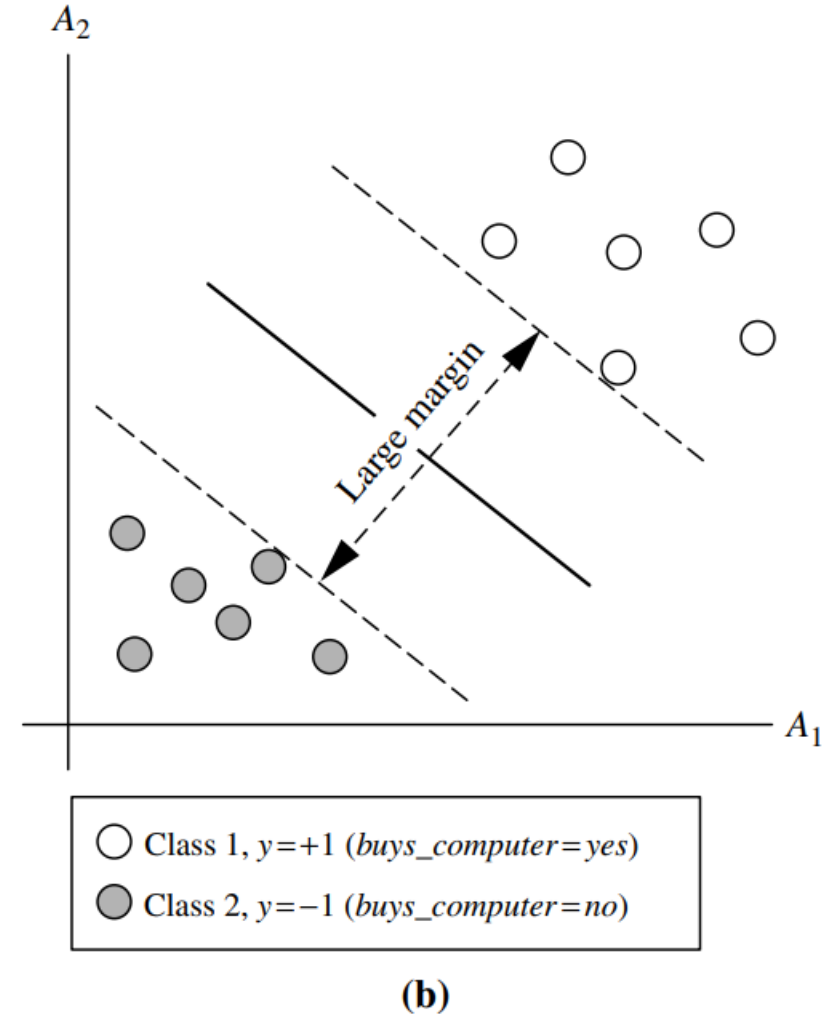
SVM: Шугаман салах боломжтой өгөгдөл

- 2 атрибуттай: A_1 болон A_2
- Атрибутуудын утга: x_1, x_2 ; $X = (x_1, x_2)$
- b -г w_0 гэж үзвэл $w_0 + w_1x_1 + w_2x_2 = 0$.
- Тусгаарлах гипер хавтгайн дээр орших ямар ч цэг дараах нөхцөлийг хангана.

$$w_0 + w_1x_1 + w_2x_2 > 0.$$

- Тусгаарлах гипер хавтгайн доор орших ямар ч цэг дараах нөхцөлийг хангана.

$$w_0 + w_1x_1 + w_2x_2 < 0.$$



SVM: Шугаман салах боломжтой өгөгдөл

- Жинг тохируулах боломжтой ба ингэснээр захын (*margin*) талуудыг тодорхойлсон гипер хавтгайг дараах байдлаар бичиж болно.

$$\begin{aligned} H_1 : w_0 + w_1 x_1 + w_2 x_2 &\geq 1 \quad \text{for } y_i = +1, \\ H_2 : w_0 + w_1 x_1 + w_2 x_2 &\leq -1 \quad \text{for } y_i = -1. \end{aligned}$$

- H_1 -с дээш эсвэл дээр нь унасан ямар ч бичлэг **+1** ангилалд
- H_2 -с доош эсвэл дээр нь унасан ямар ч бичлэг **-1** ангилалд
- Дээрх 2 томъёог нэгтгэвэл:

$$y_i = \begin{cases} 1 & \text{if } \vec{w} \bullet \vec{x}_i + b \geq 1 \\ -1 & \text{if } \vec{w} \bullet \vec{x}_i + b \leq -1 \end{cases}$$

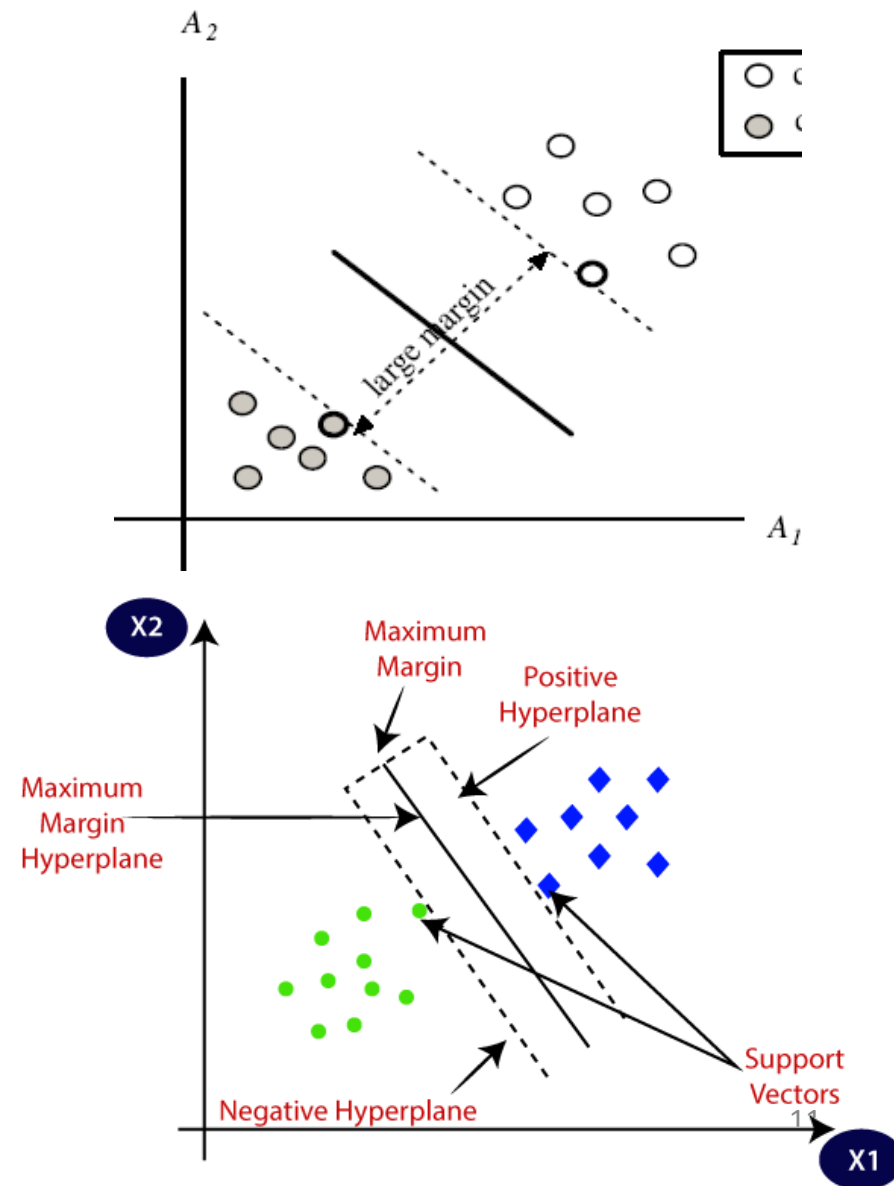
эсвэл

$$y_i(w_0 + w_1 x_1 + w_2 x_2) \geq 1, \quad \forall i.$$

SVM: Шугаман салах боломжтой өгөгдөл

- *Support vector* – Өгөгдлийн багц дахь цэг

- Өгөгдлийн багц дахь шийдвэрийн хилд хамгийн ойр байгаа цэг
- Гипер хавтгайн чиглэл болон байрлалыг тодорхойлоход хэрэглэгддэг.
- Ангилахад хамгийн хэцүү цэгүүд бөгөөд ангиллын талаар хамгийн их мэдээллийг өгдөг.



SVM: Шугаман салах боломжтой өгөгдөл

- Загвар сурна гэдэг нь w болон b -н утгыг тодорхойлохтой тэнцүү.

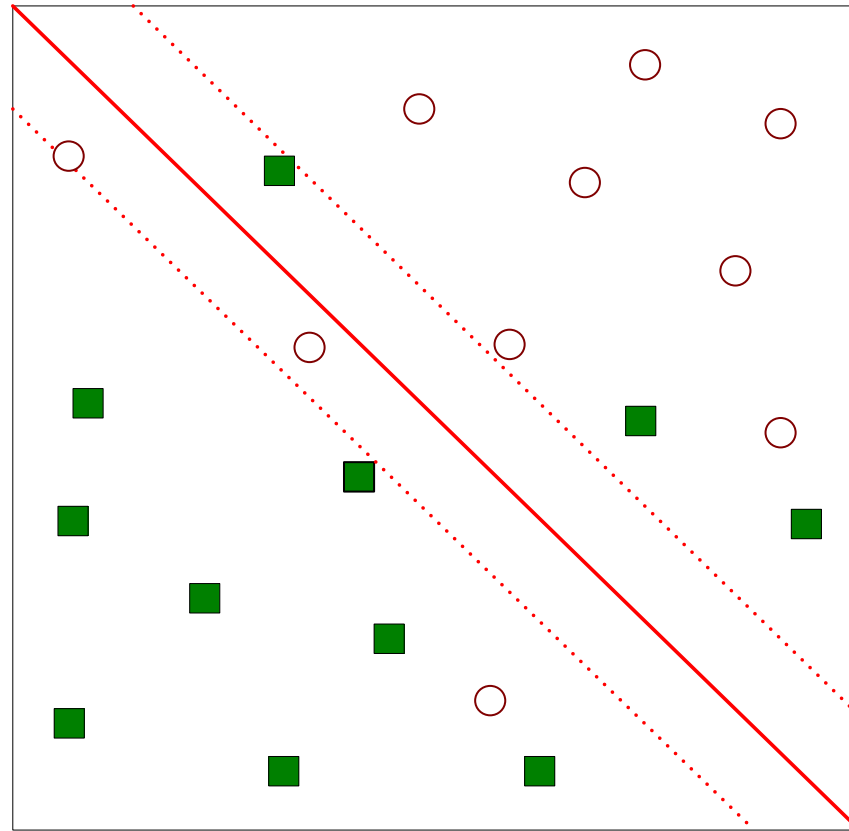
$$y_i = \begin{cases} 1 & \text{if } \vec{w} \bullet \vec{x}_i + b \geq 1 \\ -1 & \text{if } \vec{w} \bullet \vec{x}_i + b \leq -1 \end{cases}$$

$$y_i(w_0 + w_1x_1 + w_2x_2) \geq 1, \forall i.$$

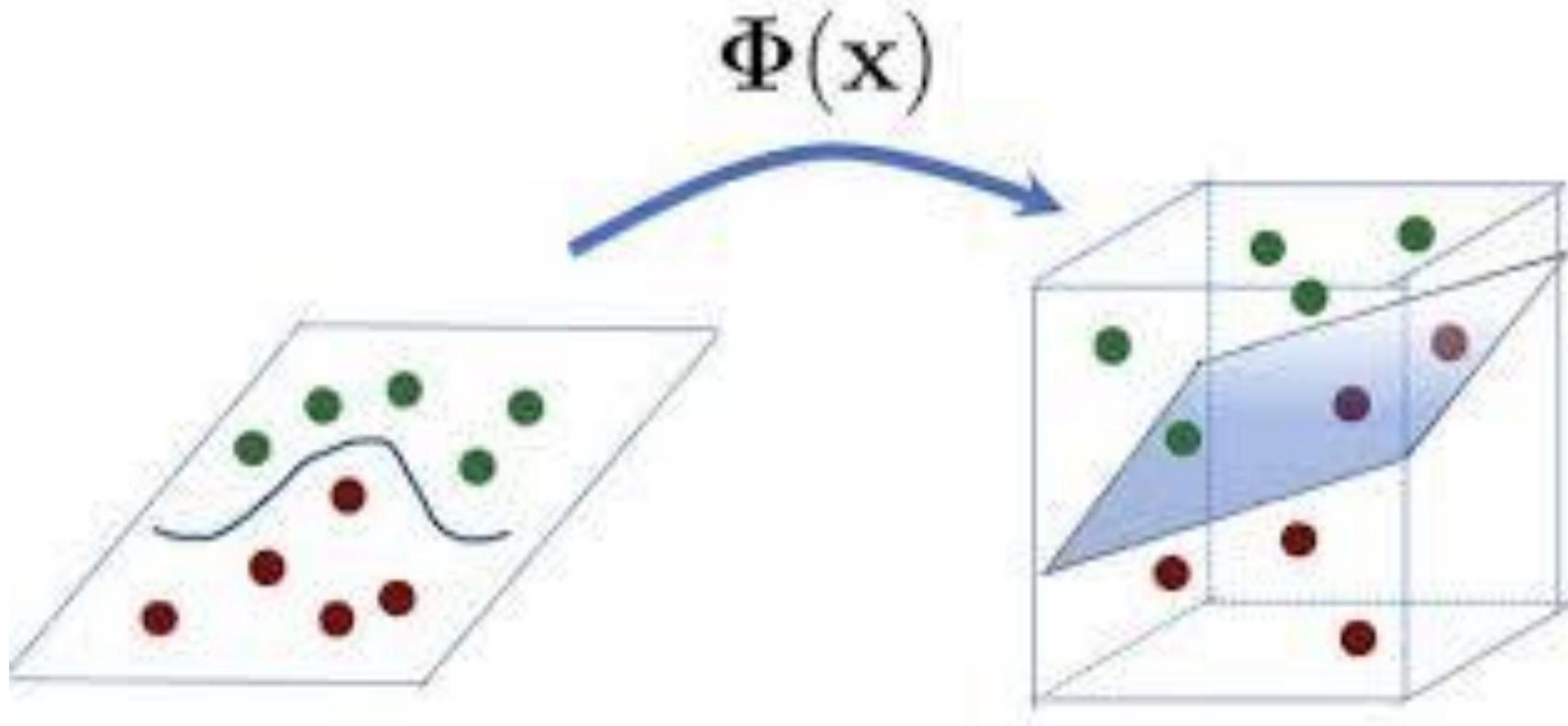
- Сургалтын өгөгдлөөс w болон b хэрхэн олох вэ?
- Хамгийн их *margin* бүхий гипер хавтгайг олох нь зорилго.
 - Энэ нь w ба b -г олох математик оновчлолын (*optimization*) асуудал болдог.
 - Энэ асуудлыг Лагранжийн үржүүлэгчийн арга (Lagrange multipliers) ашиглан шийддэг.

SVM: Шугаман салах боломжгүй өгөгдөл

- Асуудлыг шугаман салгах боломжгүй бол яах вэ?



SVM: Шугаман салах боломжгүй өгөгдөл



SVM: Шугаман салах боломжгүй өгөгдөл

- **Цөм - Kernel** (Математикийн функцүүд)

- Цөмүүд нь өгөгдлийг өндөр хэмжээст орон зайд буулгах
- Өндөр хэмжээст орон зайд үр ашигтай тооцоолох боломжийг олгодог
- Хэмжээст байдлын хараалыг багасгахад тусалдаг
- Өгөгдөл дэх шугаман бус харилцааг зохицуулах боломжийг олгодог

- **Цөмийн функцийн төрөл**

- Өгөгдлийн онцлог шинж чанар
- Шийдэж буй асуудлаас хамааран сонгоно.

$$\text{Linear : } K(w, b) = w^T x + b$$

$$\text{Polynomial : } K(w, x) = (\gamma w^T x + b)^N$$

$$\text{Gaussian RBF: } K(w, x) = \exp(-\gamma \|x_i - x_j\|^n)$$

$$\text{Sigmoid : } K(x_i, x_j) = \tanh(\alpha x_i^T x_j + b)$$

Хиймэл мэдрэлийн сүлжээ (ANN)

ANN нь хүний тархины бүтэц, үйл ажиллагаанаас санаа авсан машин сургалтын алгоритм юм.

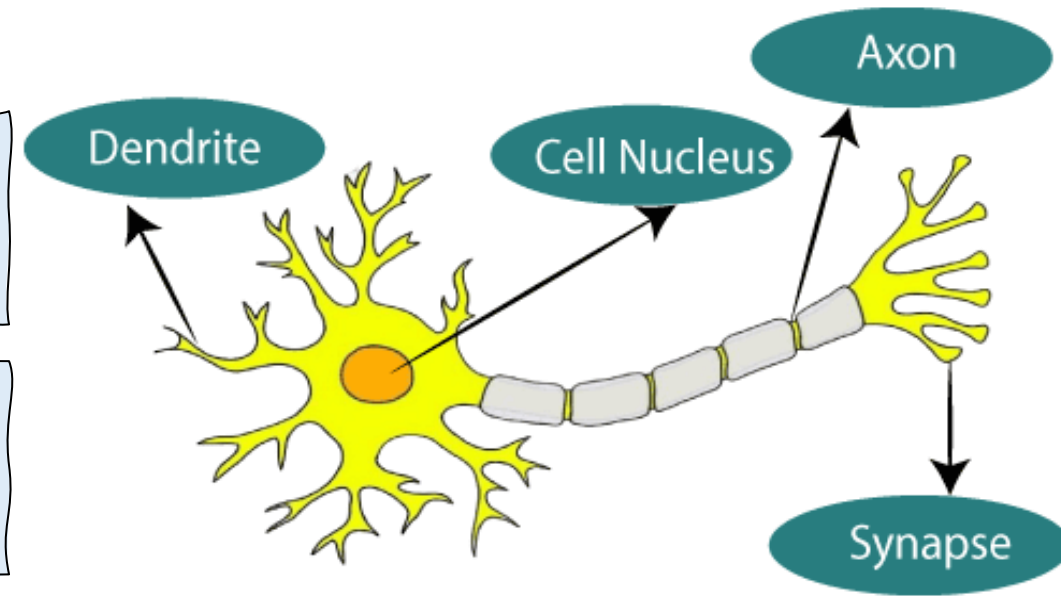
Хүний тархи нь нейрон гэж нэрлэгддэг мэдээлэл дамжуулах үүрэгтэй мэдрэлийн эсүүдээс бүрддэг.

Аксонууд нь мэдрэлийн эсүүд өдөөгдсөн үед мэдрэлийн импульсийг нэг мэдрэлийн эсээс нөгөөд дамжуулахад ашиглагддаг “*гаралт*”.

Мэдрэлийн эсийн биеийн өргөтгөл болох дендритээр дамжуулан бусад мэдрэлийн эсүүдийн аксонуудтай холбогддог “*оролт*”.

Cell – оролтын дохиог нэгтгэж боловсруулдаг “*мөв*”.

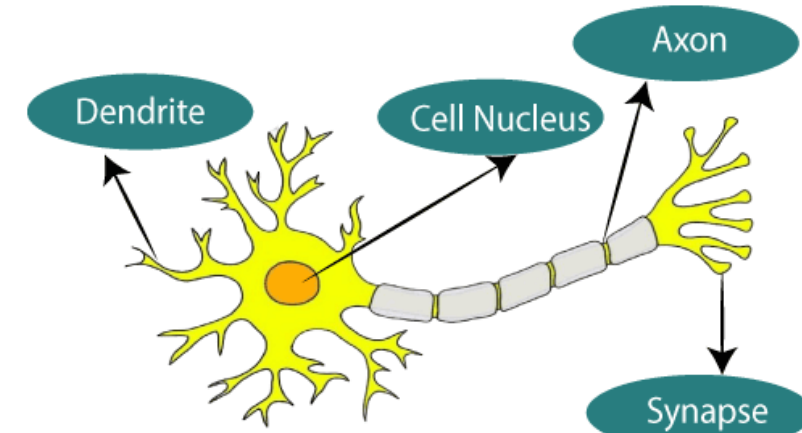
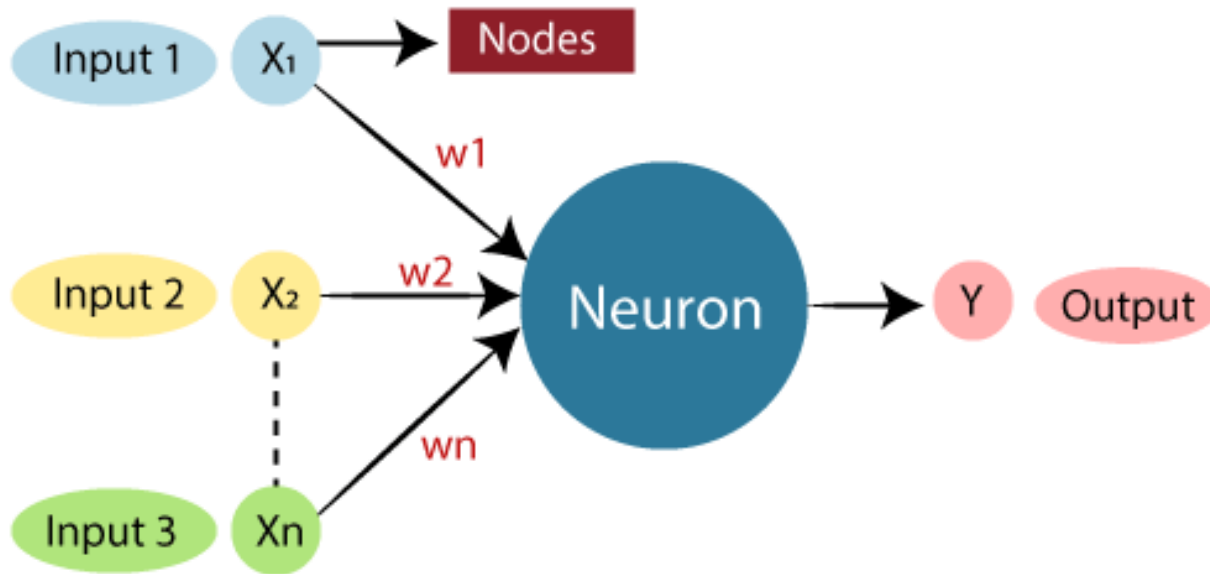
Дендрит ба аксоны хоорондох холбоо барих цэгийг синапс гэж нэрлэдэг.



86 тэрбум нейрон

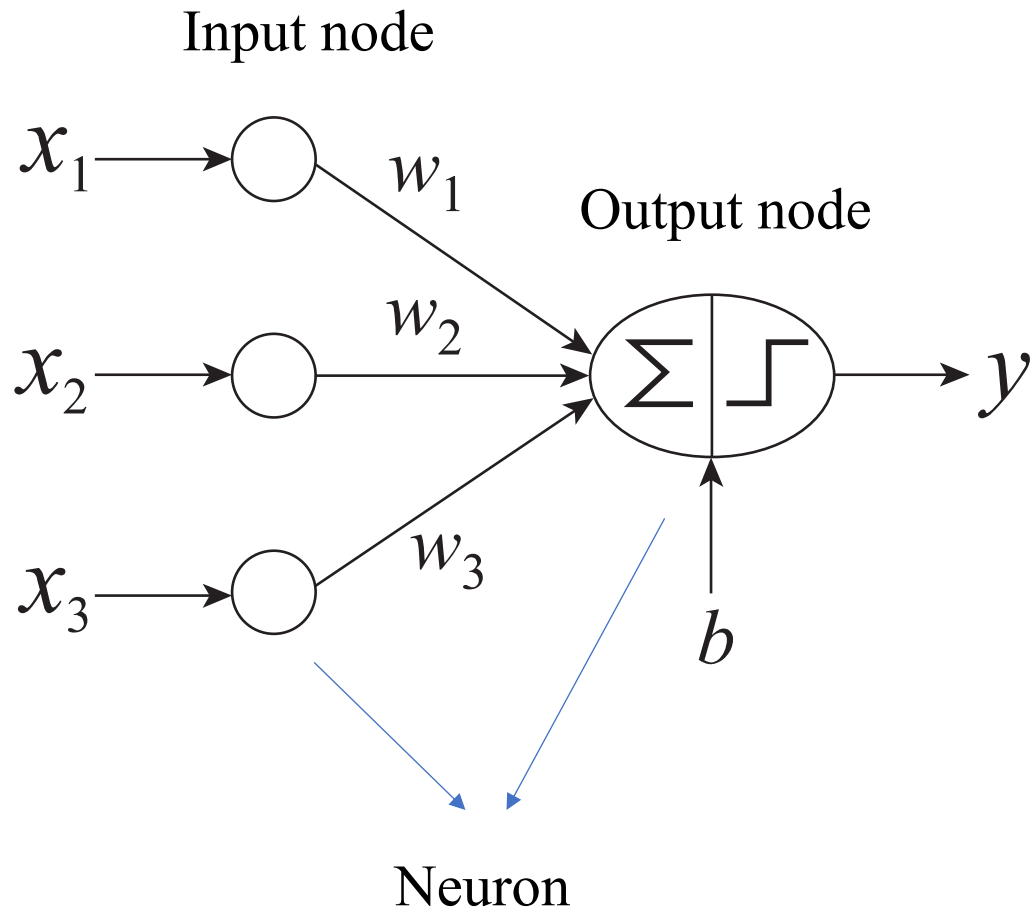
Хиймэл мэдрэлийн сүлжээ (ANN)

- *Хамгийн энгийн ANN: Перцептрон*
 - Мэдрэлийн эсүүдийн нэг давхаргаас бүрдэх ANN юм.



Biological Neural Network	Artificial Neural Network
Dendrites	Inputs
Cell nucleus	Nodes
Synapse	Weights
Axon	Output

Перцептроны үндсэн архитектур



$$y = \begin{cases} 1, & \text{if } \mathbf{w}^T \mathbf{x} + b > 0. \\ -1, & \text{otherwise.} \end{cases}$$

Перцептрон сурах дүрэм

- Жингүүдэд (*weights*) анхны утгыг олгох ($w_0, w_1, \dots, w_d$)

- *Давтах*

- (x_i, y_i) сургалтын жишээ бүрийн хувьд

- $\hat{y}_i$ тооцоолно

- Жингийн утгыг шинэчлэнэ:

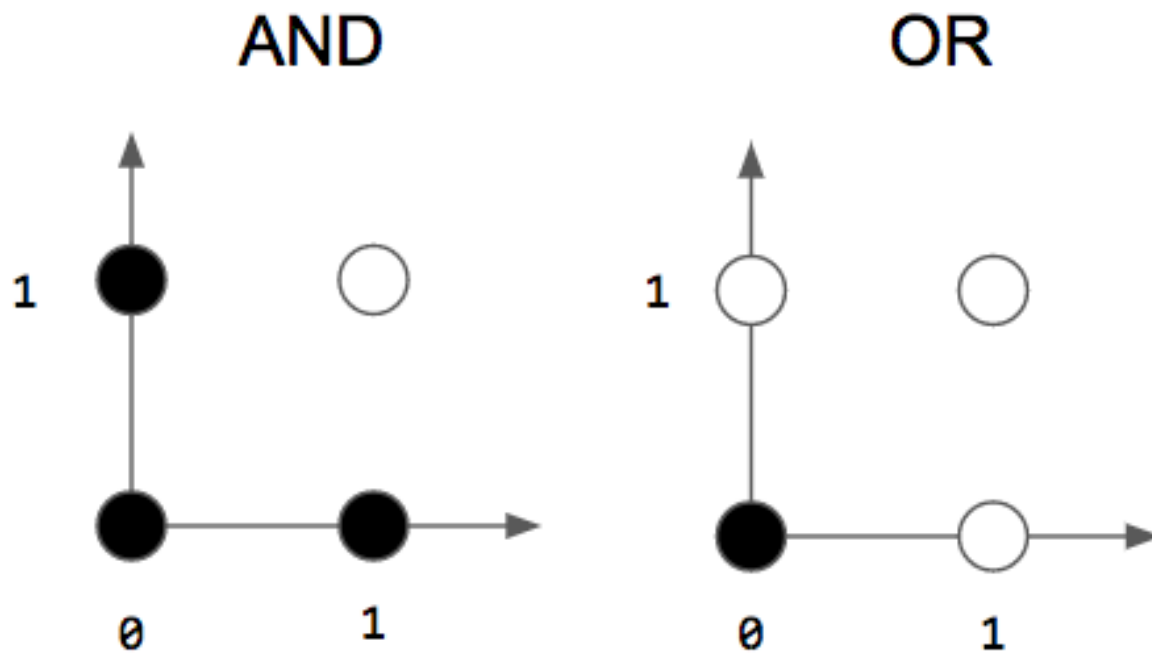
$$w_j^{(k+1)} = w_j^{(k)} + \lambda(y_i - \hat{y}_i^{(k)})x_{ij}$$

- *Зогсоох нөхцөл биелэх хүртэл*

- k : давталтын дугаар; λ : learning rate
- x_{ij} : x_i сургалтын жишээний j дахь атрибутын утга

Перцептрон сурах

- Шугаман салах боломжтой ангиллын асуудал
 - AND, OR – шугаман салах боломжтой



Шугаман бус ангиллын асуудал

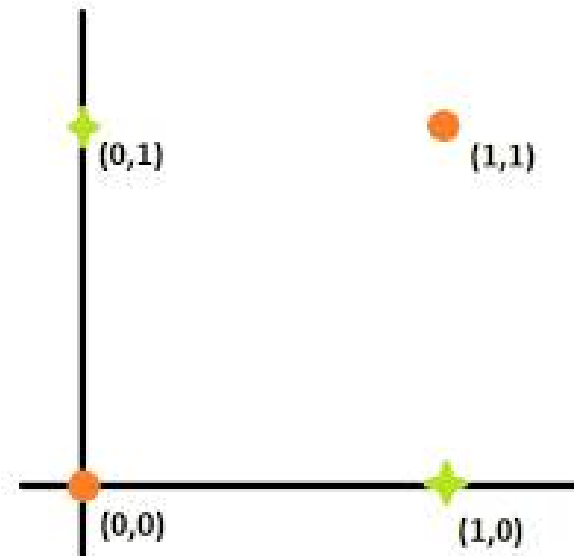
Шугаман бус байдлаар салгах шаардлагатай ангиллын асуудлын үед, ямар ч шугаман гиперхавтгай өгөгдлийг бүрэн тусгаарлаж чаддаггүй.

Иймээс ийм төрлийн бодлогод **перцептроны суралцах алгоритм тохиромжгүй** байдаг.

X_1	X_2	Y
0	0	0
0	1	1
1	0	1
1	1	0

$$Y = X_1 \oplus X_2$$

XOR Data



Multilayer Feed-Forward Neural Network

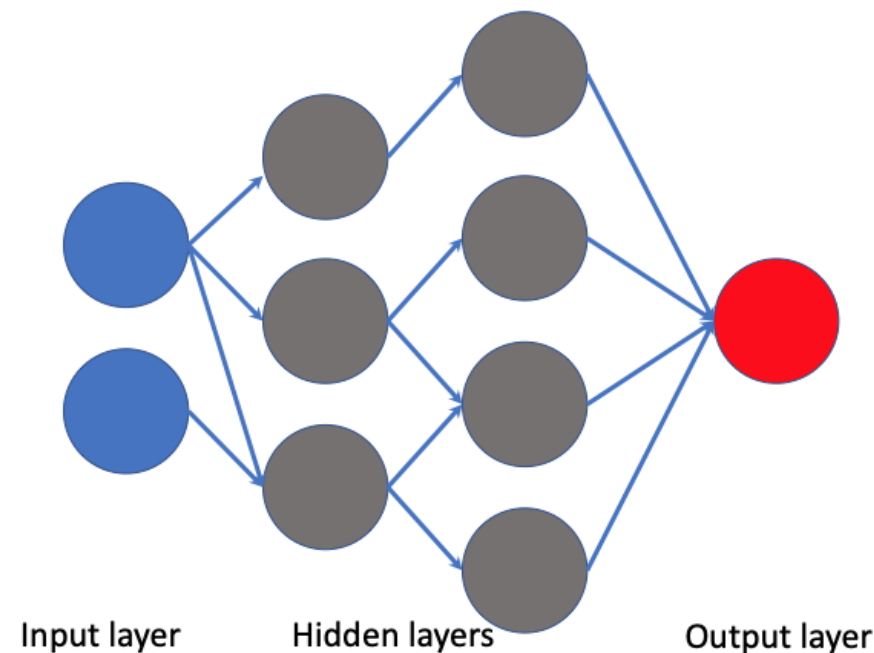
- Перцептрон загвараас илүү төвөгтэй бүтэцтэй байдаг.
- *Input layer, Hidden layer, Output layer*

Сүлжээ нь оролт гаралтын давхаргуудын хооронд хэд хэдэн завсрын давхаргыг агуулж болно.

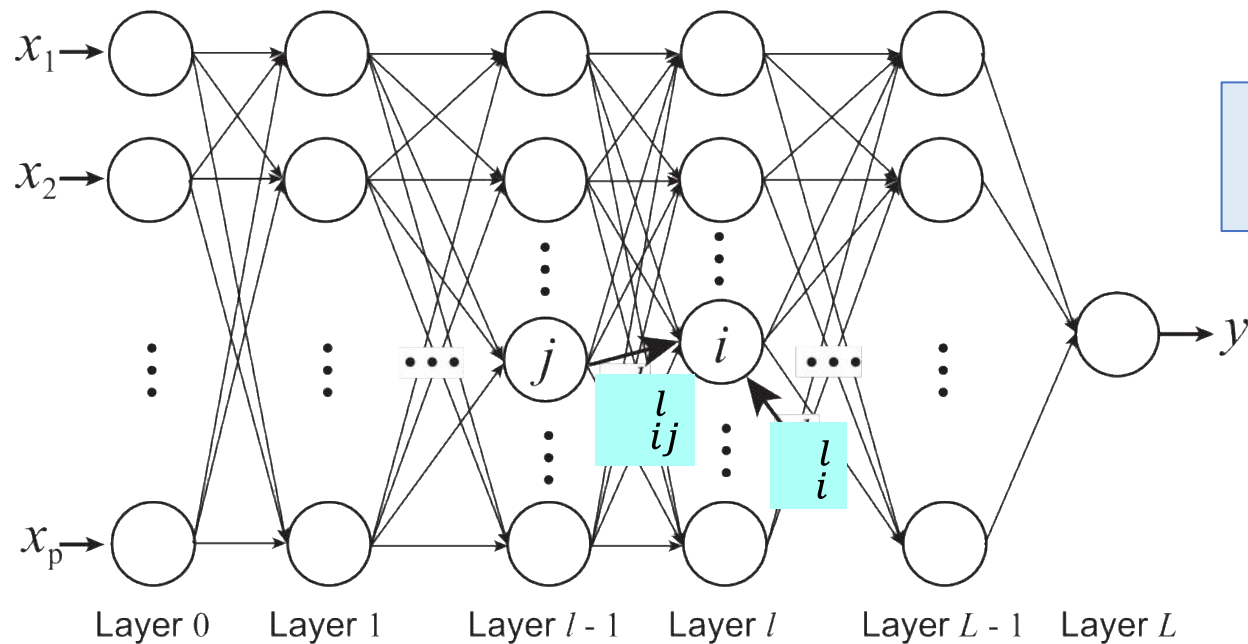
Өгөгдлийн нарийн хамаарлыг илрүүлэх гол үүргийг гүйцэтгэдэг.

Зангилаа буюу "нейрон" нь хамтдаа далд давхаргыг бүрдүүлдэг.

Зангилаа бүр нь өмнөх давхаргын зангилаануудын гаралтыг оролт болгон авч, тооцоолол хийж тодорхой гаралт үүсгэнэ.



Олон давхаргат мэдрэлийн сүлжээ



Жин нь оролтын шинж чанар бүрийн хүч эсвэл ач холбогдлыг илэрхийлдэг.

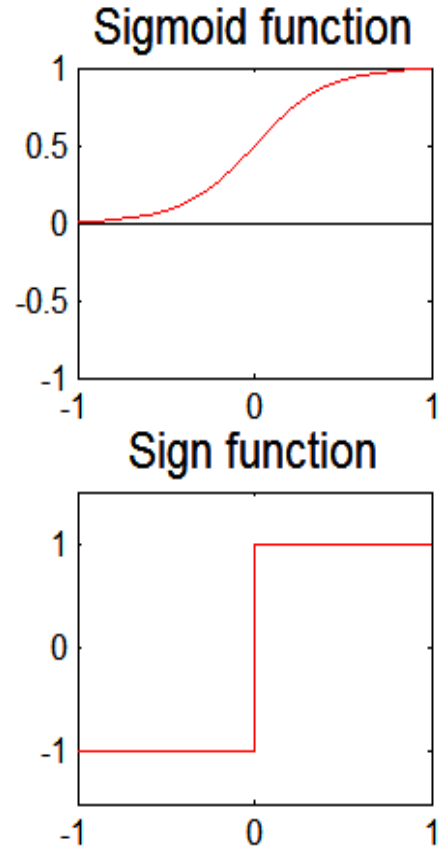
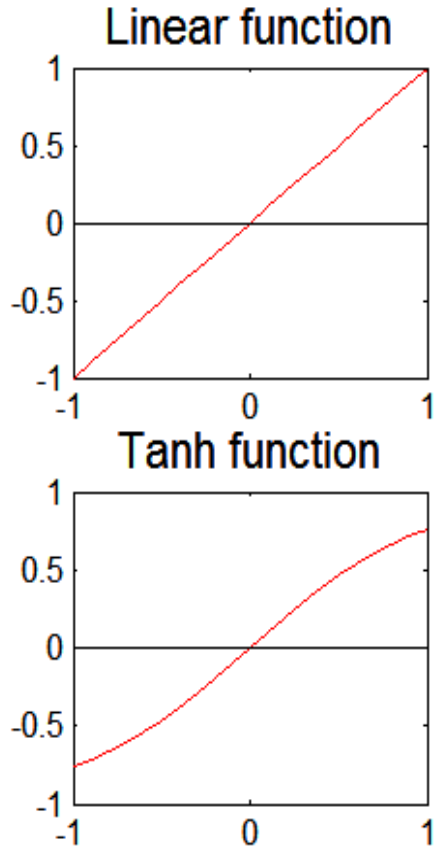
$$a_i^l = f(z_i^l) = f\left(\underbrace{\sum_j w_{ij}^l a_j^{l-1} + b_i^l}_{\text{Linear Predictor}}\right)$$

Activation value at node i at layer l

Activation Function

Linear Predictor

Activation Functions



$$a_i^l = f(z_i^l) = f\left(\sum_j w_{ij}^l a_j^{l-1} + b_i^l\right)$$

- **Sigmoid** – Гаралтын утгыг $[0, 1]$ хооронд утга
- **Tanh** – $[-1, 1]$ хооронд утга.
- **ReLU** – Эерэг утгуудыг дамжуулдаг. Сөрөг утганд 0 өгдөг.
- **Sign** – 0-с их буюу тэнцүү бол 1, эсрэг тохиолдолд -1

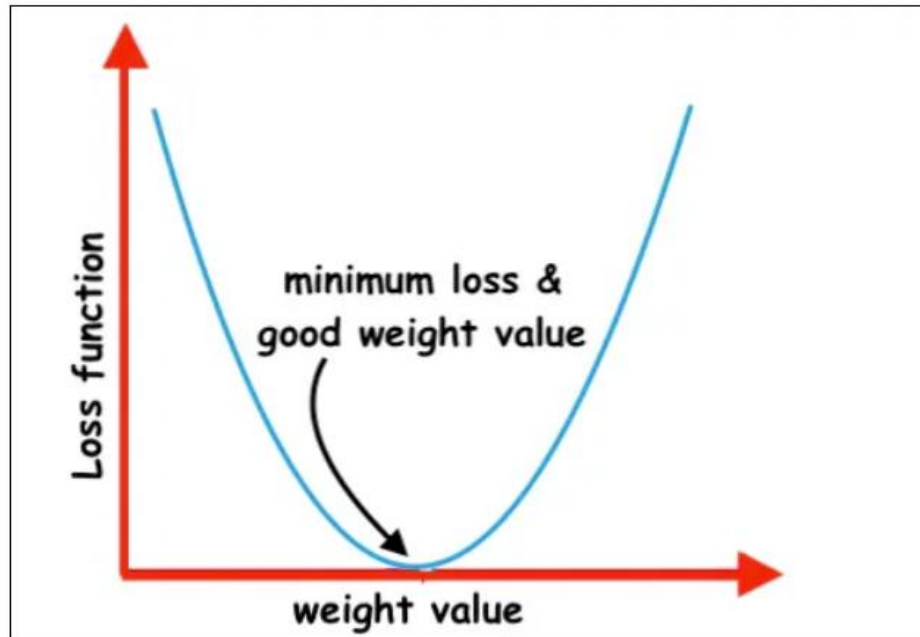
Зардлын функц (Cost function)

- Бүх сургалтын өгөгдлийн алдааг хэмжих *Loss* функц
 - Зардлын функц эсвэл алдагдлын функц

$$E(\mathbf{w}, \mathbf{b}) = \sum_{k=1}^n \text{Loss}(y_k, \hat{y}_k)$$

Squared Loss:

$$\text{Loss}(y_k, \hat{y}_k) = (y_k - \hat{y}_k)^2$$

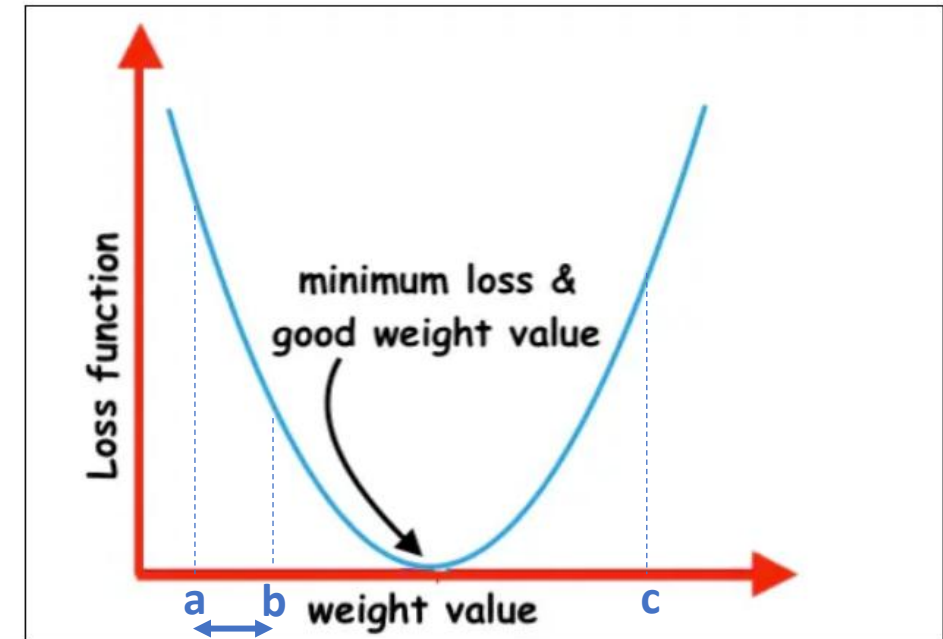


- Энэ алдагдлын функц эсвэл алдааг багасгахыг оролддог.
 - Зардлын функцыг оновчтой болгох.
- Зардлын функц дээр үндэслэн сүлжээгээр буцаж, жингийн утгыг шинэчилдэг.

Gradient Descent

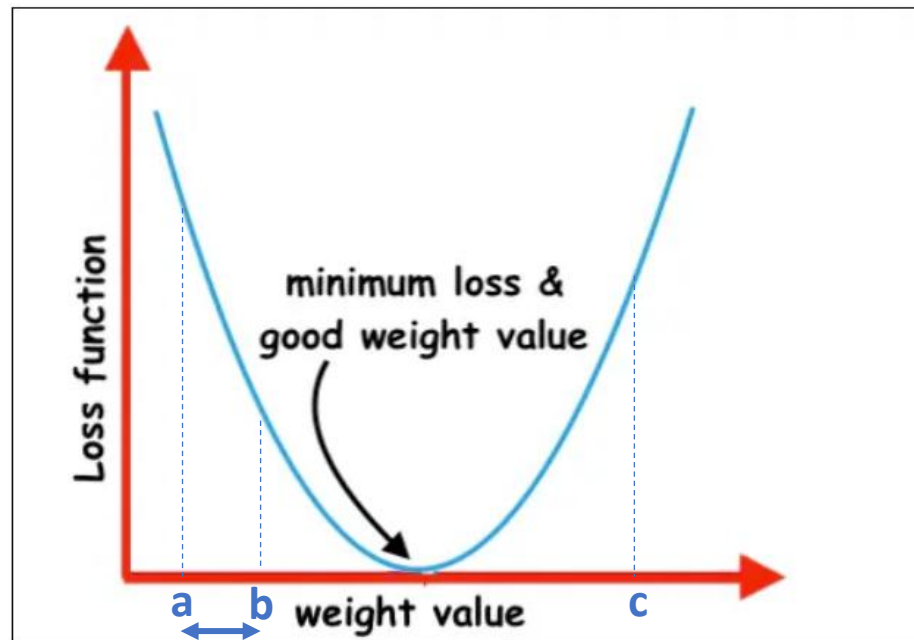
- Жингээс хамаарч алдаа хэрхэн өөрчлөгдөж байгааг шалгах хэрэгтэй.
- Үүнийг хийхийн тулд жингийн хувьд алдааны функцийн уламжлалыг олох хэрэгтэй.
- Энэ уламжлалыг **Градиент** гэж нэрлэдэг.

$$\text{Gradient} = dE/dw$$



Gradient Descent

- Хэрэв жин нэмэгдэх тусам алдагдал нэмэгдвэл градиент эерэг байх болно (*с үзэг*)
 - $\text{Градиент} > 0 \rightarrow \text{С үзгээс сөрөг } X \text{ тэнхлэгийн дагуу шилжих}$
- Хэрэв жин нэмэгдэх тусам алдагдал буурч байвал градиент сөрөг байх болно. (*а үзэг*)
 - $\text{Градиент} < 0 \rightarrow \text{а үзгээс эерэг } X \text{ тэнхлэгийн дагуу шилжих}$



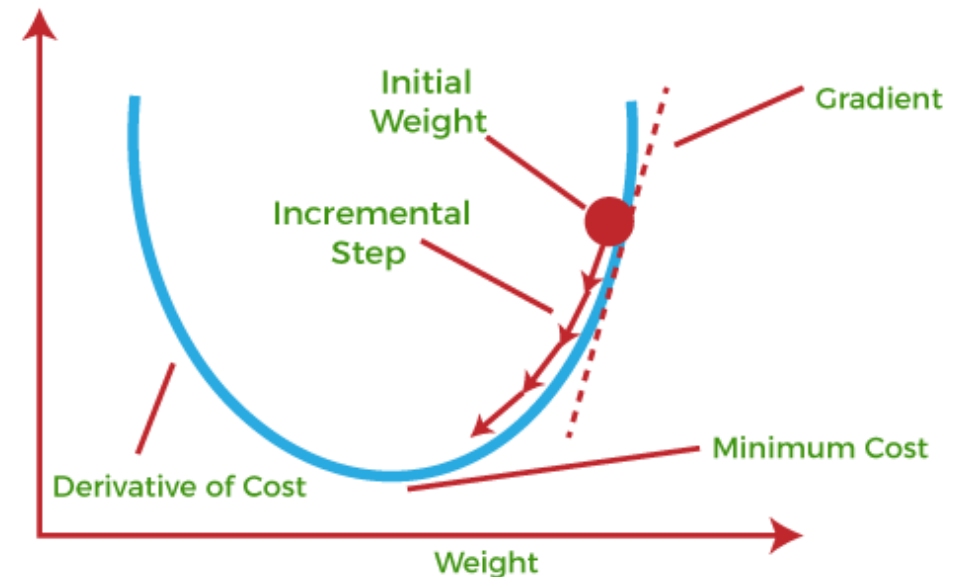
Gradient Descent

- Градиент нь алдагдлын функцийг оновчтой болгохын тулд жинг нэмэгдүүлэх эсвэл багасгах тал дээр загварыг чиглүүлдэг.
- **Загвар аль чиглэлд шилжихээ мэдсэн бол одоо хэр хэмжээгээр шилжих вэ?**
 - *Learning rate*
- **Шинэчлэгдсэн жин**

$$w_{ij}^l \leftarrow w_{ij}^l - \lambda \frac{\partial E}{\partial w_{ij}^l},$$

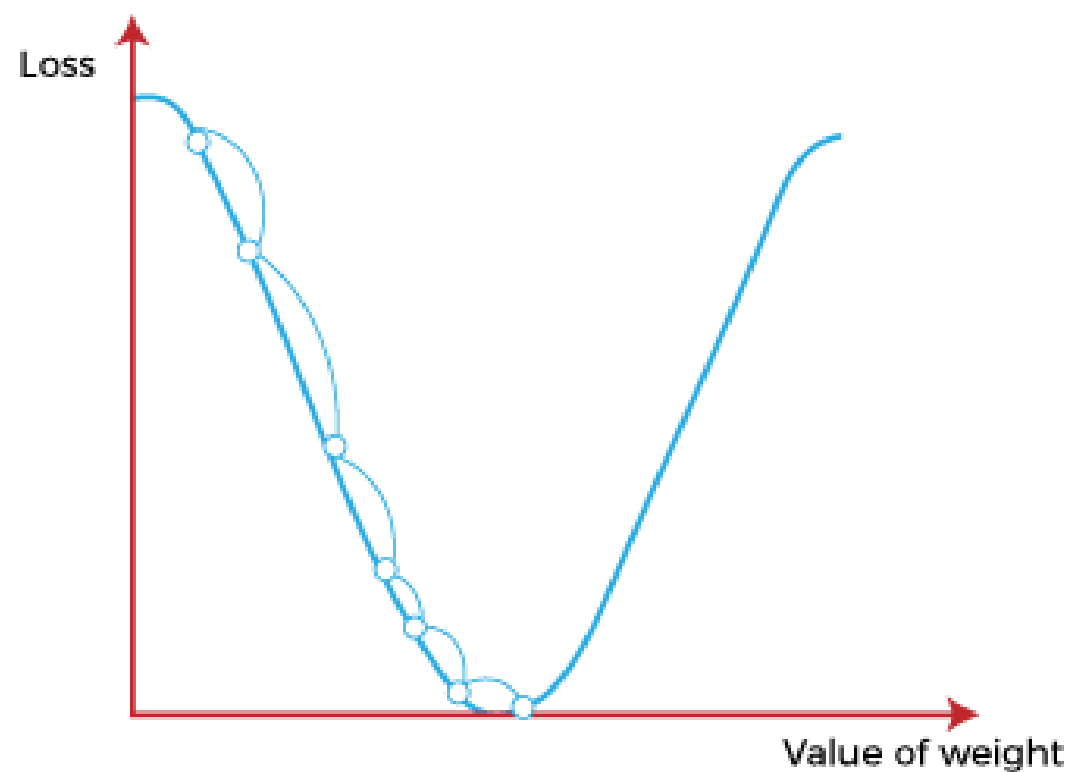
$$b_i^l \leftarrow b_i^l - \lambda \frac{\partial E}{\partial b_i^l},$$

λ : learning rate

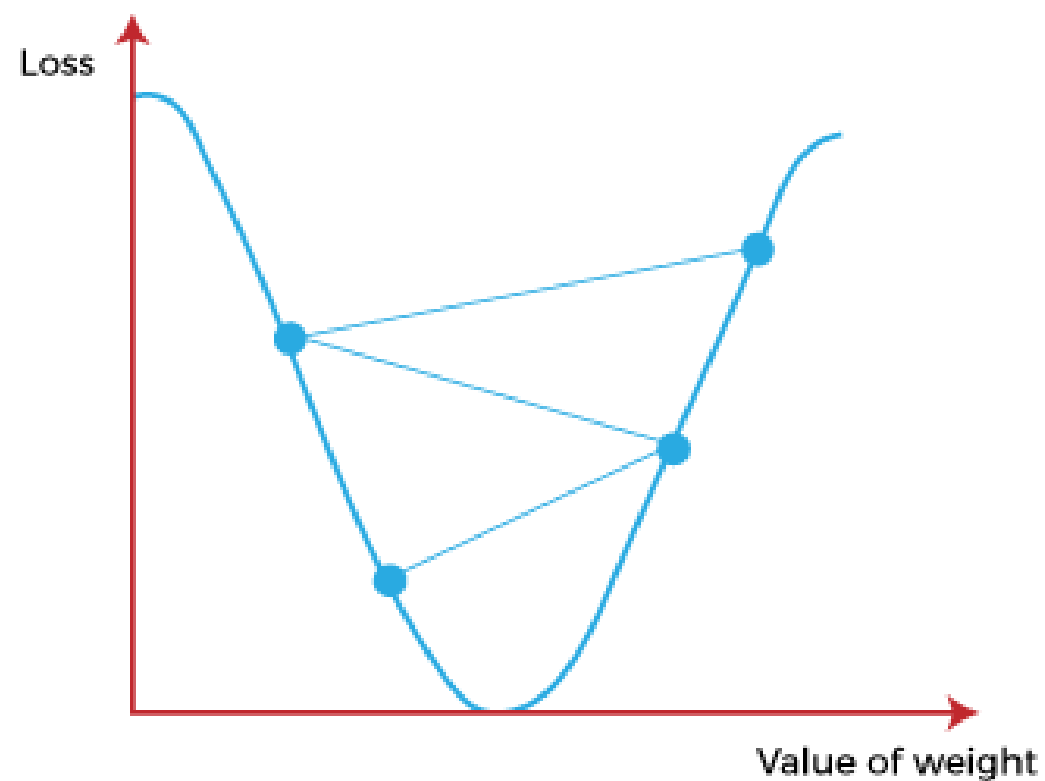


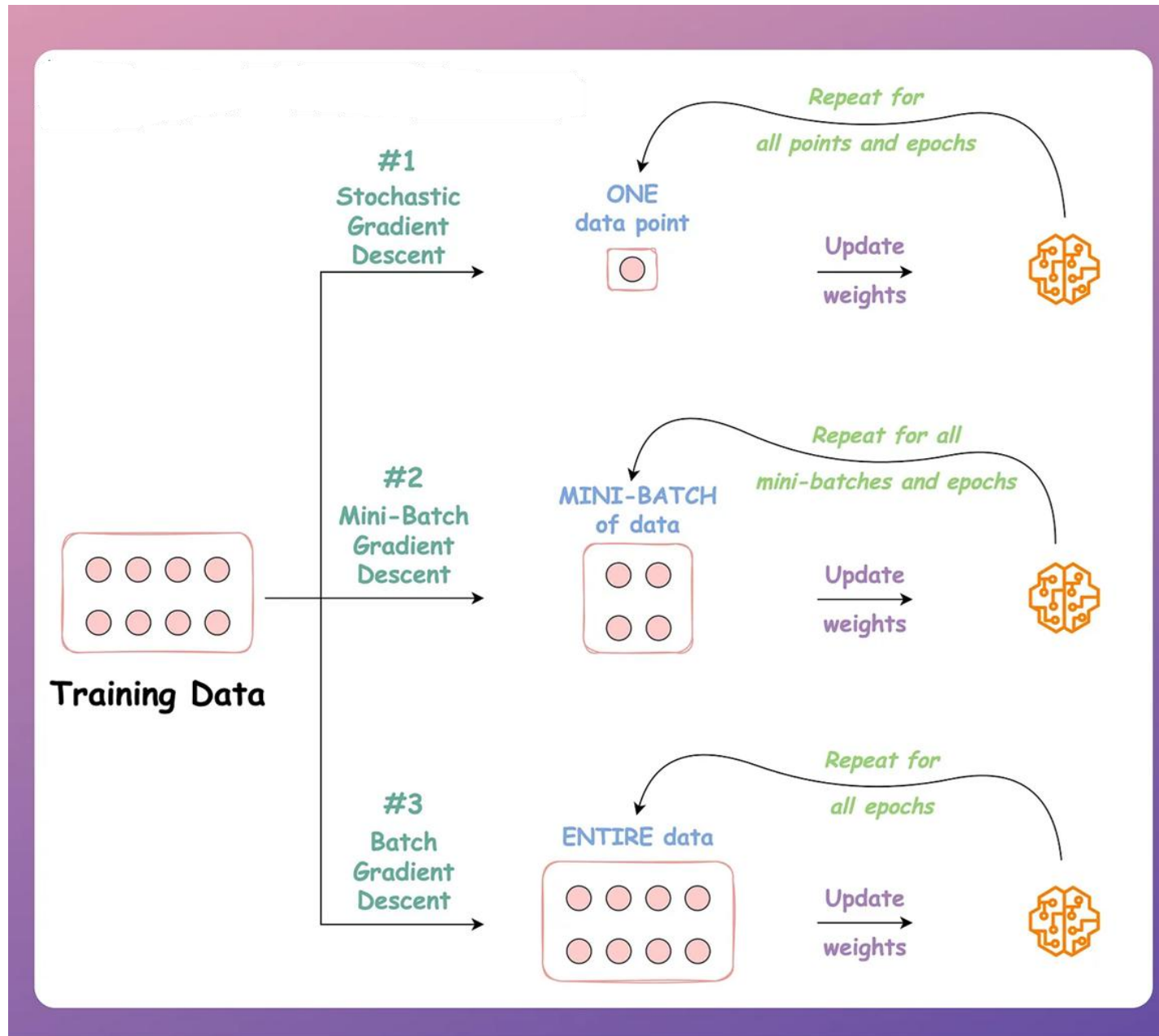
Сурах хурд

Small Learning Rate



Large Learning Rate





Градиентийг тооцоолох

$$\hat{y} = a^L$$

$$\frac{\partial E}{\partial w_{ij}^l} = \sum_{k=1}^n \frac{\partial \text{Loss}(y_k, \hat{y}_k)}{\partial w_{ij}^l}.$$

$$a_i^l = f(z_i^l) = f\left(\sum_j w_{ij}^l a_j^{l-1} + b_i^l\right)$$

- **Chain rule** (нэг жишээн дээр):

$$\frac{\partial \text{Loss}}{\partial w_{ij}^l} = \frac{\partial \text{Loss}}{\partial a_i^l} \times \frac{\partial a_i^l}{\partial z_i^l} \times \frac{\partial z_i^l}{\partial w_{ij}^l}.$$

- **Sigmoid activation function:**

$$\frac{\partial \text{Loss}}{\partial w_{ij}^l} = \delta_i^l \times a_i^l (1 - a_i^l) \times a_j^{l-1},$$

where $\delta_i^l = \frac{\partial \text{Loss}}{\partial a_i^l}.$

$$f'(x) = ((3x + 1)^5)' = 5(3x + 1)^4 (3x + 1)' = 5(3x + 1)^4 (3)$$

- keep the inside
- take derivative
of outside

multiply by
derivative of
the inside

Делта: "Энэ нейроны гаралт бага зэрэг өөрчлөгдвөл, нийт алдагдал (Loss) хэр их өөрчлөгдөх вэ?" гэдгийг хэмждэг тоо.

- Давхарга бүрийн хувьд δ_i^l үүнийг хэрхэн тооцох вэ?

Backpropagation алгоритм

- L гаралтын давхаргад:

$$\delta^L = \frac{\partial \text{Loss}}{\partial a^L} = \frac{\partial (y - a^L)^2}{\partial a^L} = 2(a^L - y).$$

- l далд давхаргын хувьд (chain rule хэрэглэх):

$$\delta_j^l = \sum_i (\delta_i^{l+1} \times a_i^{l+1} (1 - a_i^{l+1}) \times w_{ij}^{l+1}).$$

- Бүх δ -ийг олсны дараа $\partial \text{Loss} / \partial w$ -ийг бодож, градиент уналтын аргыг ашиглан жингүүдийг шинэчилнэ.

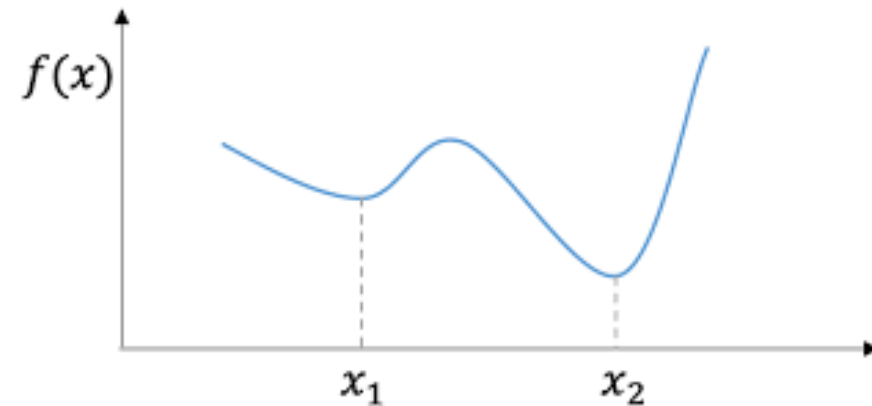
Optimizer

- *Optimizers*

- Загварыг *local minimum* оруулахаас сэргийлж сурах хурдыг автоматаар оновчтой болгож, оновчлолын үйл явцыг хурдасгах үүрэгтэй.

- Түгээмэл хэрэглэгддэг *optimizer*

- *Adam*
- *Adagrad*
- *RMSProp*
- *SGD*



БАЯРЛАЛАА

Data mining and machine learning

